

Online Explore–Then–Exploit UAV Scanning with Budgeted Return Re-Scanning

Sanggi Lee, Inseong Choi, and Soohwan Song[†]

Abstract—UAV photogrammetry efficiently reconstructs large scenes, but flight-time constraints often leave coverage gaps that require costly revisit missions. Conventional explore-then-exploit workflows address this issue through an initial scan followed by targeted re-scanning, but they typically suffer from batch reconstruction latency and manual inspection. Recent online planners reduce this delay by adapting viewpoints during flight; however, frequent short-horizon updates can limit global route quality and cause repeated trajectory modifications. We propose an *online explore-then-exploit* framework that completes initial scanning and targeted re-scanning within a single flight. During outbound exploration, the UAV follows a preplanned route, while captured imagery is incrementally processed to update a 3D proxy model, evaluate surface coverage, and identify under-scanned targets. For return re-scanning, we combine sector-level budgeted routing with global refinement using an Adaptive Large Neighborhood Search optimizer to maximize coverage gain under a fixed travel budget. By overlapping reconstruction, target extraction, and route refinement with outbound flight time, the proposed method generates high-quality return paths without requiring a prior 3D model or a separate revisit flight. Urban-scale simulations show consistent improvements in reconstruction completeness over state-of-the-art baselines under identical travel budgets.

I. INTRODUCTION

Accurate 3D models of large-scale outdoor structures are crucial for applications like infrastructure inspection [1], disaster assessment [2], and urban digital twins [3]. These tasks demand fast, low-cost, and high-fidelity data acquisition. While unmanned aerial vehicles (UAVs) are ideal for this due to their flexible mobility and diverse viewing angles, they are constrained by limited flight-time budgets and strict safety regulations. Consequently, even dense flight trajectories often fail to capture sufficient views of complex scenes, resulting in incomplete or degraded reconstructions. This incompleteness becomes costly when discovered post-flight, as it necessitates expensive and time-consuming rescanning flights.

A common strategy in UAV scanning is the *explore-then-exploit* approach [4], [5], [6], [7]: the UAV first executes a simple coverage trajectory to build an initial model, followed by an additional flight to recapture poorly reconstructed regions. By targeting missing areas identified during the initial scan, this two-stage paradigm substantially improves capture efficiency and reconstruction completeness. Furthermore, because the UAV follows precomputed trajectories within

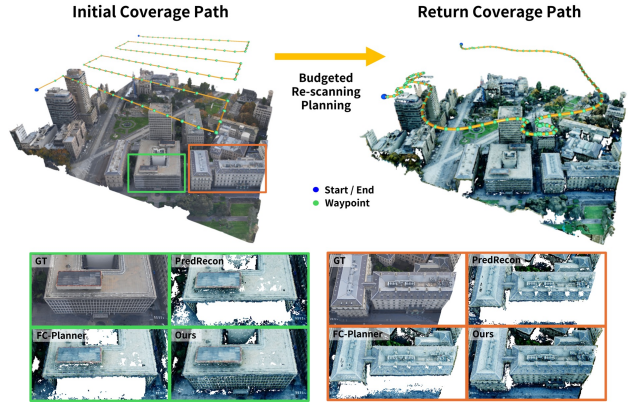


Fig. 1. Overview of our online explore-then-exploit method. (*top*) Our method follows an initial coverage path while continuously analyzing the 3D model online to simultaneously generate a return coverage path for rescanning under-scanned areas. (*bottom*) Under equal path-length budgets, zoomed-in comparisons against the ground truth (GT) demonstrate that our method outperforms PredRecon [11] and FC-Planner [12], reliably recovering missing structures and minimizing holes.

known free space, operational risks remain low. However, in practice, this workflow is severely bottlenecked by batch 3D reconstruction latency and the need for manual inspection between flights. This disconnect breaks the continuous acquisition loop, resulting in significant delays.

Recently, online planning methods [8], [9], [10], [11] have emerged, leveraging real-time 3D modeling feedback to continuously update scanning trajectories during flight. By enabling single-trial scanning without prior models, they partially alleviate the drawbacks of the classical workflows. However, most online planners rely on frequent onboard replanning driven by real-time perception. The required dense online reconstruction is highly compute-intensive, often exceeding the limits of embedded UAV platforms. Furthermore, strict time constraints force these planners to rely on short-horizon local updates, yielding globally suboptimal trajectories regarding coverage and flight budgets. Finally, dynamic trajectory modifications involving altitude changes compromise UAV stability and safety, rendering their direct application to practical 3D scanning non-trivial.

Therefore, we propose a novel *online explore-then-exploit* paradigm bridging the strengths of conventional workflows and online planning. Unlike classical pipelines relying on offline reconstruction and manual inspection to schedule a second flight, our method integrates exploration and quality assessment within the first flight. As the UAV traverses an initial coverage route, it streams imagery to a ground server.

The authors are with the Department of Computer Science and Artificial Intelligence, Dongguk University, Seoul 04620, Republic of Korea

[†]Corresponding author: songsh@dongguk.edu

This work was supported by the MSIT of the Republic of Korea through the National Research Foundation (NRF) (No. RS-2026-25489947) and the Commercialization Promotion Agency for R&D Outcomes (COMPA) (No. 2710086167).

The server incrementally updates a 3D model and computes per-surface coverage scores, automatically identifying low-coverage targets without manual review. Concurrently, it computes a budget-feasible return route to rescan these targets. By executing the optimized path during the return flight, the UAV completes both the initial scan and targeted re-scan in a single mission. This design allows return-path refinement to overlap with the outbound flight, reducing the need for continuous trajectory updates during exploration.

To achieve efficiency, we propose a sector-wise online refinement strategy by partitioning the workspace into ordered sectors. At each sector boundary, the server updates a 3D model, identifies visible re-scan candidates, and constructs a budget-feasible local sub-route. These sub-routes form a global route skeleton, which is continuously optimized by an Adaptive Large Neighborhood Search-based (ALNS) destroy-repair algorithm [13] utilizing the computation time available before reaching the next milestone. Upon reaching the terminal waypoint, the server finalizes a globally refined return route that balances coverage gain and flight cost. As shown in Fig. 1, our approach yields higher-quality reconstructions than prior baselines [11], [12].

Our contributions are summarized as follows:

- We extend the offline explore-then-exploit paradigm into a fully online framework. By coupling real-time MVS reconstruction with budgeted return-phase re-scan planning, our approach enables complete, high-fidelity 3D modeling in a single flight.
- We propose a sector-wise online informative planning strategy for re-scanning. Using incremental geometric updates per sector, it continuously optimizes a mid-flight return path to maximize coverage of under-scanned regions within a cumulative budget.
- We validate our method through urban-scale simulations, demonstrating its effectiveness across multiple scenes and travel budgets. The source code is made publicly available¹.

II. RELATED WORK

A. Offline UAV Coverage Path Planning

A dominant baseline in aerial 3D scanning is executing pre-planned coverage patterns (e.g., zig-zag or grid flights) that prioritize operational simplicity. To address complex geometries, many methods adopt an explore-then-exploit paradigm [4], [5], [6]. Within this framework, several works formulate viewpoint selection as a submodular optimization problem, maximizing information gain based on surface coverage to efficiently select complementary views [4], [5].

Recent offline planners tightly couple route design with reconstruction quality [7], [3], [14]. Zhang et al. [7] optimize reconstruction fidelity and path smoothness from a rough proxy. Extending this, Xiong et al. [3] propose a budget-constrained planner for urban geometry and texture co-capture. For complex scenes with occlusions, sampling-based methods [12], [15], [16] are frequently employed to naturally

handle collision constraints. To further improve efficiency in cluttered environments, FC-Planner [12] utilizes a skeleton-guided framework that extracts a scene’s structural backbone to generate fast, non-redundant coverage paths.

Despite their effectiveness, these multi-flight workflows are severely bottlenecked by the latency of offline batch processing. In contrast, our method integrates incremental model updating and budget-feasible re-scan planning entirely into a single flight. By leveraging online feedback, we successfully eliminate the post-hoc delays.

B. Online View Path Planning

To mitigate the latency of offline workflows, recent studies explore online and active planning to dynamically adapt viewpoints during flight. A standard formulation is next-best-view selection, where candidate views are iteratively evaluated using an information-gain objective within a receding horizon. This gain is computed across various representations, including frontier-based unknown space [17], volumetric occupancy [18], [1], surface observation adequacy [19], and recently, neural rendering-based utility [20].

In aerial applications, several systems tightly couple online 3D modeling with planning to autonomously acquire informative views. Song et al. [8], [9] utilize online *multi-view stereo* (MVS) to continuously plan paths using reconstructability cues. Kuang et al. [10] achieve real-time urban reconstruction by combining top-down initial routing, SLAM-based height estimation, and coverage prediction. Similarly, PredRecon [11] leverages predictive modeling to rapidly guide flights toward missing areas. Collectively, these online planners demonstrate the potential to eliminate repeated site visits and significantly improve reconstruction completeness within a single mission.

Despite their promise, online planners [8], [9], [11], [21] face practical barriers. Dense reconstruction and frequent re-planning exceed onboard compute limits, while short-horizon updates yield globally inefficient routes. To reduce this short-horizon limitation, our method incrementally builds a global return-route skeleton online and refines it during outbound flight, reducing the need for active trajectory updates.

C. Budget-Constrained Informative Routing

Selecting and ordering informative viewpoints under a travel budget is closely related to *orienting* [22] and *routing-with-profits* [23] problems, where the goal is to maximize collected reward subject to length/time constraints. Among practical solvers, *Adaptive Large Neighborhood Search* (ALNS) [13], [24] is particularly effective: it repeatedly applies destroy-repair operators with adaptive operator weighting and metaheuristic acceptance to quickly improve solutions. In robotics, resource-bounded informative planning is also studied in monitoring and exploration [10], [25].

Our approach solves a budget-constrained routing problem in an online setting, where coverage deficits and view utilities evolve alongside incremental 3D updates. Consequently, we couple sector-wise candidate generation and local routing with ALNS-based global refinement, leveraging flight time to yield an optimized return route under strict budgets.

¹<https://lisk1233.github.io/explore-then-exploit>

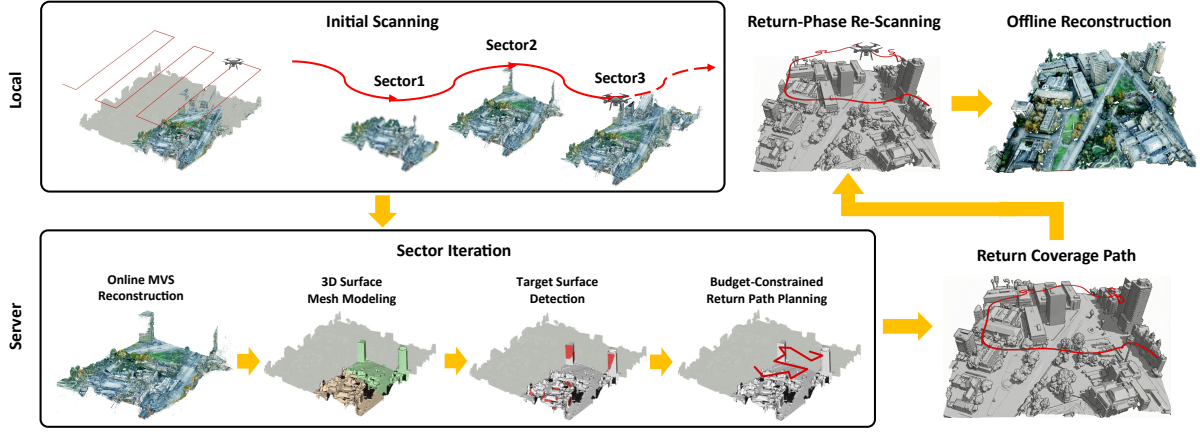


Fig. 2. Sector-wise online reconstruction and budgeted return-path planning. During the initial outbound scan (top), the UAV follows a preplanned coverage route partitioned into ordered sectors. At each boundary, the ground server performs a sector iteration (bottom): updating the online MVS 3D reconstruction, detecting under-scanned regions, and refining a budget-constrained return path. The UAV executes this path during its return flight, after which the captured high-resolution images are processed into the final 3D model.

III. PROPOSED METHOD

We address the *budgeted UAV re-scanning* problem for efficient, high-fidelity 3D reconstruction. Limited flight resources dictate strategic revisits to under-scanned regions. To ensure safe, unobstructed traversal in practical urban scanning, the UAV maintains a fixed altitude above the tallest structure. This fixed-altitude setting is commonly adopted in practical UAV photogrammetry for maintaining stable clearance above structures. We maximize quality by executing a two-phase mission within one flight. During outbound exploration, the UAV continuously captures images along an initial trajectory, ξ^{init} . The subsequent return re-scan targets only the most informative regions via a budget-feasible path, ξ^{rtn} , optimized on a server using real-time feedback. Decoupling data acquisition from global optimization eliminates offline processing latency and heavy onboard computation.

As illustrated in Fig. 2, the exploring UAV saves high-resolution images locally while streaming low-resolution versions to the server. We assume a stable wireless link; when brief dropouts occur, the system buffers imagery and performs updates at sector boundaries. The server runs an online MVS pipeline to maintain a 3D point cloud and assess surface coverage via a lightweight proxy model. To bound latency, the workspace is partitioned into sectors. Crossing a sector boundary triggers the server to incrementally update the model, identify under-scanned surfaces, and refine a tentative return plan. These background updates naturally overlap with the UAV’s flight time.

Upon reaching the outbound terminal waypoint, the server finalizes ξ^{rtn} via a one-shot, budget-constrained global optimization [26]. This sequence is converted into a dynamically feasible trajectory, which the UAV executes during its home-ward flight to cover the detected gaps.

A. Sector-based Hybrid Online Processing

To ensure bounded online latency in large-scale environments, our framework processes mapping data incrementally

during the exploration phase, deferring the final global re-scan optimization until the outbound flight terminates.

We partition the workspace into K rectangular sectors, $\mathcal{S} = \{\mathcal{S}_1, \dots, \mathcal{S}_K\}$, ordered according to the outbound coverage trajectory. Specifically, each sector $\mathcal{S}_k$ is an axis-aligned rectangle corresponding to the footprint of a specific sweep segment of ξ^{init} . By advancing to $\mathcal{S}_{k+1}$ only upon crossing these boundaries, we ensure stable per-sector data volumes and predictable server-side processing times. As illustrated in Fig. 3, this parallelized workflow guarantees that by the end of the outbound phase, only the final sector remains to be processed before the server initiates the one-shot global optimization for the return flight.

B. Online 3D Model Reconstruction

We maintain a dynamic 3D point cloud from streaming images using an online MVS pipeline inspired by MVS-GS [27], [28]. Unlike MVS-GS, which uses these points primarily to initialize and train 3D Gaussian Splatting, we accumulate the point cloud as a geometric substrate for surface modeling. Continuously updated during flight, this model provides immediate geometric cues for identifying under-scanned regions.

Let $\{I_t\}$ denote the sequence of streamed low-resolution images and $T_t \in SE(3)$ represent the estimated camera pose at time t . Poses are tracked in real-time by a DROID-SLAM-style [29] frontend. Concurrently, we incrementally construct a keyframe co-visibility graph $G = (\mathcal{K}, \mathcal{E})$, where nodes $\mathcal{K}$ represent keyframes and edges $\mathcal{E}$ link co-visible pairs. This graph structure facilitates periodic global optimization and defines multi-view neighborhoods for depth inference. For each new keyframe $\kappa \in \mathcal{K}$, we select its five most co-visible neighbors as source frames. We then estimate the reference depth map D_κ using a cascade cost-volume MVS network, MVSFormer++ [30], which refines depth across multiple scales for real-time efficiency.

Each depth map D_κ is back-projected ($\mathcal{B}$) into 3D space and fused into a global point cloud $\mathcal{P} = \bigcup_{\kappa \in \mathcal{K}} \mathcal{B}(D_\kappa, T_\kappa)$.

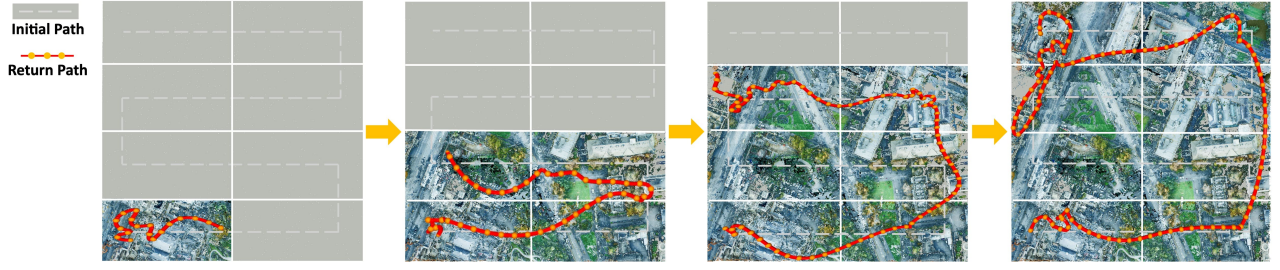


Fig. 3. Online exploration progress visualization on the Atlas City scene. Each panel shows the incremental 3D reconstruction and the updated return coverage path (red line) as the flight follows the initial coverage route (dashed line). The white grid denotes sector partitions; reconstructed sectors gradually expand over time, while unreconstructed sectors remain blank.

To suppress outliers, we employ (i) confidence-based filtering and (ii) multi-view geometric filtering via depth reprojection [31]. Finally, to mitigate trajectory drift, global bundle adjustment [29] is executed every 30 keyframes over the entire graph G . The optimized poses are then propagated to the fused point cloud via efficient rigid transformations, ensuring global consistency with minimal online overhead.

C. Surface Coverage Evaluation

1) *Incremental Surface Proxy Generation*: To facilitate efficient visibility queries and coverage assessment, we maintain a lightweight surface proxy that is updated incrementally as sectors are processed. For each sector point cloud $\mathcal{P}_k \subseteq \mathcal{P}$, we first apply statistical outlier removal [32] followed by voxel-grid downsampling.

Within each sector's bounding box, augmented by a small overlap margin, we reconstruct a watertight mesh $\mathcal{M}_k$ using Poisson surface reconstruction [33]. We then prune low-density vertices and uniformly resample the surface [34] to obtain a set of surface samples $\mathcal{X}_k$. Each sample $x_j \in \mathcal{X}_k$ is defined by its 3D position, an estimated normal vector n_j , and a coverage score $\text{Cov}(x_j)$ derived from processed frames. The local proxy components $(\mathcal{M}_k, \mathcal{X}_k)$ are integrated into the global model $\mathcal{M}$ and surface-sample set $\mathcal{X}$, respectively. This proxy $(\mathcal{M}, \mathcal{X})$ serves as the geometric substrate for coverage scoring and viewpoint utility evaluation.

2) *Coverage Score and Target Surface Extraction*: We quantify the observation quality of each surface by assigning a coverage score $\text{Cov}(x_j) \in [0, 1]$ to all $x_j \in \mathcal{X}$. Low scores indicate insufficient observation, while high scores signify saturated coverage reliable for MVS reconstruction.

Let $\mathcal{V}_{\text{img}}$ represent the set of camera viewpoints where each viewpoint is $v = (p, R)$ with position $p \in \mathbb{R}^3$ and orientation $R \in SO(3)$. Visibility is determined via frustum culling and ray casting against $\mathcal{M}$, with the visible subset denoted by $\text{Vis}(v, \mathcal{X}) \subseteq \mathcal{X}$. For each visible sample $x_j \in \text{Vis}(v, \mathcal{X})$, we compute a geometric visibility score:

$$S_{\text{geo}}(x_j, v) = s_{\text{dist}}(x_j, v) s_{\text{inc}}(x_j, v) s_{\text{ctr}}(x_j, v), \quad (1)$$

where the constituent factors are defined as:

$$s_{\text{dist}}(x_j, v) = \max\left(0, 1 - \frac{\|x_j - p\|}{d_{\text{max}}}\right), \quad (2)$$

$$s_{\text{inc}}(x_j, v) = |\hat{r}_j^\top n_j|, \quad \hat{r}_j = \frac{x_j - p}{\|x_j - p\|}, \quad (3)$$

$$s_{\text{ctr}}(x_j, v) = \max\left(0, 1 - \frac{\alpha_j}{\alpha_{\text{fov}}}\right). \quad (4)$$

Here, d_{max} is the sensing range, α_j is the angular offset from the optical axis, and α_{fov} is the half field-of-view angle. The distance factor s_{dist} penalizes observations nearing the maximum range d_{max} , while the incidence factor s_{inc} weights observations by the absolute cosine between the viewing ray and surface normal to mitigate grazing-angle degradation. Finally, the centering factor s_{ctr} penalizes points near the camera frustum periphery.

We accumulate per-frame visibility scores to obtain the raw coverage:

$$\text{Cov}(x_j) = \sum_{v \in \mathcal{V}_{\text{img}}: x_j \in \text{Vis}(v, \mathcal{X})} S_{\text{geo}}(x_j, v), \quad (5)$$

and normalize it to the range $[0, 1]$ as $\min(1, \text{Cov}(x_j))$.

Under-scanned target regions are subsequently extracted as the set $\mathcal{X}_{\text{tar}} = \{x_j \in \mathcal{X} \mid \text{Cov}(x_j) < \tau_{\text{cov}}\}$, where $\tau_{\text{cov}} \in (0, 1)$ is a predefined threshold. This target set $\mathcal{X}_{\text{tar}}$ directly identifies missing surface regions, providing the necessary inputs to evaluate candidate viewpoints and plan the return coverage path in Section III-D.

D. Coverage Path Planning

Our goal is to generate a *budget-feasible* return coverage path, ξ^{rtn} , that revisits under-scanned regions to capture the target surfaces, $\mathcal{X}_{\text{tar}}$, while the UAV executes an initial out-bound route, ξ^{init} . To achieve this, we continuously update a 3D model online, assess surface coverage, and incrementally refine the return plan on the server. The primary challenge lies in balancing viewpoint informativeness (expected coverage gain) with travel cost under a strict flight budget. We address this using a two-level strategy: a *sector planner* that builds a local coverage path within a sector budget, and a *global planner* that aggregates these local plans, refining the mission-level route via an ALNS-based optimizer [13].

A route is an ordered sequence of views, $\pi = [v_0, \dots, v_m]$, with a length defined as $L(\pi) = \sum_{i=0}^{m-1} \|p_{i+1} - p_i\|$. Let $\mathcal{V}$

Algorithm 1 Global Coverage Path Planning with Online ALNS Refinement

Require: Sector set $\mathcal{S} = \{\mathcal{S}_1, \dots, \mathcal{S}_K\}$ (outbound order), initial proxy model $(\mathcal{M}, \mathcal{X})$, global length budget B , sector length budget $\bar{B}$, start view s

Ensure: Executable continuous path ξ with $L(\xi) \leq B$

```

1:  $\mathcal{V} \leftarrow \emptyset$ 
2:  $\pi \leftarrow [s]$ 
3: for  $k \leftarrow 1$  to  $K$  do
4:    $\mathcal{M} \leftarrow \text{Update3DModel}((\mathcal{M}, \mathcal{X}), \mathcal{S}_k)$ 
5:    $\mathcal{V}_k \leftarrow \text{SampleViewpoints}((\mathcal{M}, \mathcal{X}), \mathcal{S}_k)$ 
6:    $\pi_k \leftarrow \text{SectorCoveragePlanning}((\mathcal{M}, \mathcal{X}), \mathcal{V}_k, \bar{B})$ 
7:    $\mathcal{V} \leftarrow \mathcal{V} \cup \mathcal{V}_k$ 
8:    $\pi \leftarrow \text{MergeIntoRoute}(\pi, \pi_k)$ 
9:    $B_k \leftarrow \bar{B} \times k$ 
10:  repeat
11:     $\pi \leftarrow \text{ALNSUpdate}(\mathcal{V}, \pi, (\mathcal{M}, \mathcal{X}), B_k)$ 
12:  until  $\text{NextSectorUpdateTriggered}()$ 
13: end for
14:  $\xi \leftarrow \text{GetContinuousPath}(\pi)$ 
15: return  $\xi$ 

```

and $\mathcal{V}_k$ denote the global and sector-level view candidate sets, respectively. The total flight budget is B , and each sector receives an equal budget, $\bar{B} = B/K$. During the outbound flight, we define intermediate milestones $\mathcal{G} = \{g_k\}_{k=1}^K$, where g_k marks the boundary of sector k (the entry point into $\mathcal{S}_{k+1}$). These milestones serve strictly to time-bound the online refinement process.

Algorithm 1 outlines the global coverage planner, which constructs a mission-level return path within the total budget B by iterating through sectors in outbound order (lines 3–13). The algorithm maintains an accumulated global candidate set, $\mathcal{V}$, and a current global route, π . For each sector $\mathcal{S}_k$, the server first integrates newly arrived measurements to update the current 3D proxy model, $(\mathcal{M}, \mathcal{X})$ (line 4). Based on this updated model, we sample a sector-level candidate pool, $\mathcal{V}_k$ (line 5). Specifically, viewpoints are uniformly sampled within $\mathcal{S}_k$ and filtered using a visibility constraint: candidates seeing fewer target surface samples than a threshold τ_{vis} are discarded to prevent degenerate views.

Given $\mathcal{V}_k$ and the budget $\bar{B}$, the sector planner (Algorithm 2) computes a feasible sector route, π_k (line 6). The global planner then aggregates these candidates and routes, updating $\mathcal{V} \leftarrow \mathcal{V} \cup \mathcal{V}_k$ and $\pi \leftarrow \pi \cup \pi_k$ (lines 7–8). This produces an increasingly comprehensive global route skeleton as the outbound flight progresses.

Meanwhile, the global route is iteratively refined via an ALNS-based optimizer until $\text{NextSectorUpdateTriggered}$ is activated (line 12). This event occurs when the UAV reaches milestone g_{k+1} , prompting an update for sector $\mathcal{S}_k$. This re-optimizes viewpoint selection and ordering across the accumulated candidate set $\mathcal{V}$, constrained by B_k (the cumulative budget up to sector k), while preserving the best-found discrete route π (lines 9–12). Once the outbound phase concludes,

Algorithm 2 Sector Coverage Planning

Require: Proxy model $(\mathcal{M}, \mathcal{X})$, candidate set $\mathcal{V}_k$, sector budget $\bar{B}$

Ensure: Feasible sector route π_k^* with $L(\pi_k^*) \leq \bar{B}$

```

1:  $\pi_k \leftarrow []$ 
2:  $\{C_1, \dots, C_M\} \leftarrow \text{DBSCAN}(\mathcal{V}_k)$ 
3: while  $\mathcal{V}_k \neq \emptyset$  do
4:    $\text{UpdateCoverageScore}((\mathcal{M}, \mathcal{X}), \pi_k)$ 
5:   for each  $v \in \mathcal{V}_k$ : compute  $Q(v)$ 
6:    $(v^*, C_{v^*}) \leftarrow \text{SelectMaxGain}(\mathcal{V}_k)$ 
7:    $\bar{\pi}_k \leftarrow \text{InsertAtBestPos}(\pi_k, v^*)$ 
8:   if  $L(\bar{\pi}_k) \leq \bar{B}$  then
9:      $\pi_k \leftarrow \bar{\pi}_k$ 
10:  end if
11:   $\mathcal{V}_k \leftarrow \mathcal{V}_k \setminus C_{v^*}$ 
12: end while
13:  $\pi_k^* \leftarrow \text{2OptTSP}(\pi_k)$ 
14: return  $\pi_k^*$ 

```

the final discrete route π is converted into a dynamically feasible continuous trajectory ξ via MINCO optimization [26], yielding the executable return coverage path, ξ^{rtn} (lines 14–15).

1) *Sector Planner:* Algorithm 2 computes a local route within sector $\mathcal{S}_k$ under the budget $\bar{B}$. To prevent the repeated selection of near-duplicate viewpoints, we cluster the candidates in $\mathcal{V}_k$ into $\{C_1, \dots, C_M\}$ using the DBSCAN algorithm [35] (line 2). By clustering nearby viewpoints based on a distance threshold and minimum neighbor count, and treating isolated samples as noise, DBSCAN inherently promotes spatial diversity.

The planner then greedily expands a partial route π_k , alternating between updating coverage scores and selecting the most informative remaining viewpoint. We evaluate a candidate viewpoint v by its expected coverage gain:

$$Q(v) = \sum_{x_j \in \text{Vis}(v, \mathcal{X}_{\text{tar}})} \{\text{Cov}^+(x_j, v) - \text{Cov}(x_j)\}, \quad (6)$$

where $\text{Vis}(v, \mathcal{X}_{\text{tar}})$ denotes the set of target surface samples visible from v . Here, $\text{Cov}(x_j)$ and $\text{Cov}^+(x_j, v)$ are the coverage scores of x_j before and after observing it from v , respectively. For each visible target sample $x_j \in \text{Vis}(v, \mathcal{X}_{\text{tar}})$, we approximate the update as

$$\text{Cov}^+(x_j, v) = \min(1, \text{Cov}(x_j) + S_{\text{geo}}(x_j, v)). \quad (7)$$

We define the coverage gain as the sum of the resulting coverage-score improvements induced by selecting v .

The SelectMaxGain routine selects the candidate maximizing $Q(v)$ and returns its corresponding cluster C_v (line 6). This chosen view is inserted into the current route at the position that minimizes the incremental length increase. The insertion is accepted only if the resulting route remains within the budget, $L(\pi_k) \leq \bar{B}$ (lines 8–10). Following insertion, all candidates within the selected cluster are removed ($\mathcal{V}_k \leftarrow \mathcal{V}_k \setminus C_v$, line 11) to further encourage spatial diversity.

Once the greedy loop terminates, a fast 2-opt refinement is applied to shorten the tour before returning π_k^* (lines 13–14).

2) *ALNS-based Global Coverage Update*: After appending π_k , Algorithm 1 refines the global route online using *ALNSUpdate* (lines 10–12). In our implementation, this function applies an ALNS utilizing a standard destroy-repair framework [13]. ALNS iteratively destroys a subset of viewpoints in the current route and repairs it by reinserting candidates that offer high marginal coverage gains, all while respecting the remaining budget. The destroy operators are chosen adaptively, ensuring robust online refinement as the candidate set $\mathcal{V}$ expands.

At sector k , the cumulative budget B_k remains available for refinement until the next-sector update event is triggered. In practice, this corresponds to the UAV reaching the next milestone g_{k+1} , so the route accumulated up to $\mathcal{S}_k$ is refined while the UAV traverses the following sector. During each ALNS iteration, starting from an initial feasible route π_{init} (the current π), we search for a route that maximizes the objective value:

$$U(\pi) = \sum_{v \in \pi} Q(v) \quad \text{s.t.} \quad L(\pi) \leq B_k$$

We then update the best-so-far solution, π_{best} , accordingly. The procedure consists of three core steps:

Destroy. We remove a fraction β of viewpoints from the current π_{best} to generate a large neighborhood. Specifically, we first eliminate low-contribution viewpoints with a small $Q(v)$, followed by a small number of random viewpoints. This promotes diversification and helps avoid local optima. We set $\beta = 10\%$, resulting in a degraded route, π_{deg} . The adaptive weight update rule for operator selection is adopted directly from [13].

Repair. We reconstruct π_{deg} by greedily inserting views from the pool $\mathcal{V}$. For a given candidate $v = (p, R)$, we calculate the insertion cost of the best-position in the current route:

$$\Delta L(v | \pi) = \min_i \left(\|p_i - p\| + \|p - p_{i+1}\| - \|p_i - p_{i+1}\| \right)$$

Candidates are then ranked by an efficiency score, $e(v | \pi) = \frac{Q(v)}{\Delta L(v | \pi)}$. We iteratively insert the highest-efficiency viewpoint at its optimal position, accepting the insertion only if the route remains feasible under the budget ($L(\pi) \leq B_k$).

Acceptance. The repaired route is accepted if it improves upon π_{best} . Optionally, a simulated-annealing acceptance rule can be applied to permit occasional non-improving moves. Whenever a route is accepted, we apply a lightweight intra-route local search (2-opt) [36] to minimize $L(\pi)$, thereby freeing up budget slack for additional insertions.

IV. EXPERIMENTS

A. Experimental Setups

Implementation Details. We implement the full pipeline in a realistic simulation environment based on Unreal Engine 4 with the AirSim plugin². All planning and reconstruction modules run on a dedicated server equipped with an AMD

²<https://github.com/Microsoft/AirSim>

TABLE I
RECONSTRUCTION QUALITY EVALUATION UNDER THREE BUDGETS.

Method	Metric	Atlas City			Turkey City			EU City		
		700	600	500	400	300	200	600	500	400
Parallel Sweep	Precision	99.76	99.58	99.42	99.99	99.94	99.90	99.42	99.68	99.77
	Recall	74.59	69.85	68.75	68.41	54.76	39.17	89.37	74.70	73.89
	F-score	85.36	82.11	81.29	81.24	70.75	56.28	94.13	85.40	84.90
	Coverage	97.89	95.88	92.83	97.63	90.87	86.24	95.78	94.62	92.71
FC-Planner	Precision	99.81	99.59	99.57	99.98	100.00	99.99	99.67	99.84	99.90
	Recall	73.53	68.29	67.95	70.18	55.19	42.83	91.79	75.36	74.40
	F-score	84.68	81.02	80.77	82.48	71.12	59.97	95.58	85.89	85.29
	Coverage	97.74	96.15	92.13	99.52	97.15	92.16	96.56	95.37	94.17
PredRecon	Precision	99.29	99.83	99.41	99.95	99.81	99.68	99.59	99.67	99.60
	Recall	75.22	71.75	69.15	69.16	60.29	41.42	89.33	86.05	82.58
	F-score	85.59	83.41	81.57	81.75	75.18	58.52	94.18	92.36	90.29
	Coverage	98.35	96.25	91.61	95.53	91.19	85.81	95.78	94.95	92.94
Ours	Precision	99.83	99.83	99.77	99.97	100.00	99.96	99.68	99.71	99.66
	Recall	78.97	76.12	75.47	76.23	70.41	45.35	93.99	91.98	90.76
	F-score	88.18	86.38	85.93	86.50	82.64	62.40	96.74	95.69	95.01
	Coverage	98.71	96.60	94.19	99.58	97.67	89.59	98.92	96.64	93.47

Ryzen 9 5900X CPU and RTX 4080S GPU, while the UAV executes only low-level flight control and image capture.

Baselines. We benchmark the proposed method on three large-scale urban scenes with diverse geometry and occlusion patterns: Atlas City (Fig. 1, $281 \times 281 \times 96 \text{ m}^3$), Turkey City (Fig. 4, $132 \times 130 \times 17 \text{ m}^3$), and EU City (Fig. 4, $115 \times 85 \times 40 \text{ m}^3$). We compared our method with the following representative scanning planners under identical path-length budgets and the same fixed-altitude flight setting:

- **Parallel Sweep**: is a baseline executing a lawnmower pattern, similar to the initial trajectory, until exhausting the travel budget.
- **PredRecon** [11]: is an online planning method that incrementally reconstructs the 3D scene in real time and predicts the structure of unscanned regions to continuously update the flight path.
- **FC-Planner** [12]: is an offline method that plans a coverage path based on a prior model; in our experiments, we use the ground-truth (GT) 3D model as the prior.

Evaluation Protocol. A simulated UAV is equipped with an RGB camera featuring an $(80^\circ, 50^\circ)$ field of view and 1280×720 resolution. It cruises at a constant speed of $v_{\text{max}} = 1.5 \text{ m/s}$. We conducted experiments under varying maximum travel-distance budgets B^* , measured in meters, according to each scene scale. Our method allocates a fixed budget B^{init} of 350 (Atlas City), 140 (Turkey City), 300 (EU City) to generate the initial path, reserving the remaining budget ($B = B^* - B^{\text{init}}$) for the return trajectory. We set the parameters to $\alpha_{\text{fov}} = 25.25$, $\tau_{\text{cov}} = 0.1$, and $\tau_{\text{vis}} = 5$.

Metrics. For fair comparison, all methods use the same offline COLMAP pipeline [37] to generate final 3D models. We evaluate reconstruction quality under the flight budget by rigidly aligning each point cloud to a uniformly sampled GT reference [38]. We report Precision, Recall, and F-score using a distance threshold of $\tau_{\text{dist}} = 0.1 \text{ m}$ [38]. Additionally, coverage is measured as the fraction of GT surface points visible from at least $\tau_{\text{vis}} = 5$ viewpoints.

B. Experimental Results

Table I reports the reconstruction quality against path-length budgets across three urban scenes. Because Precision

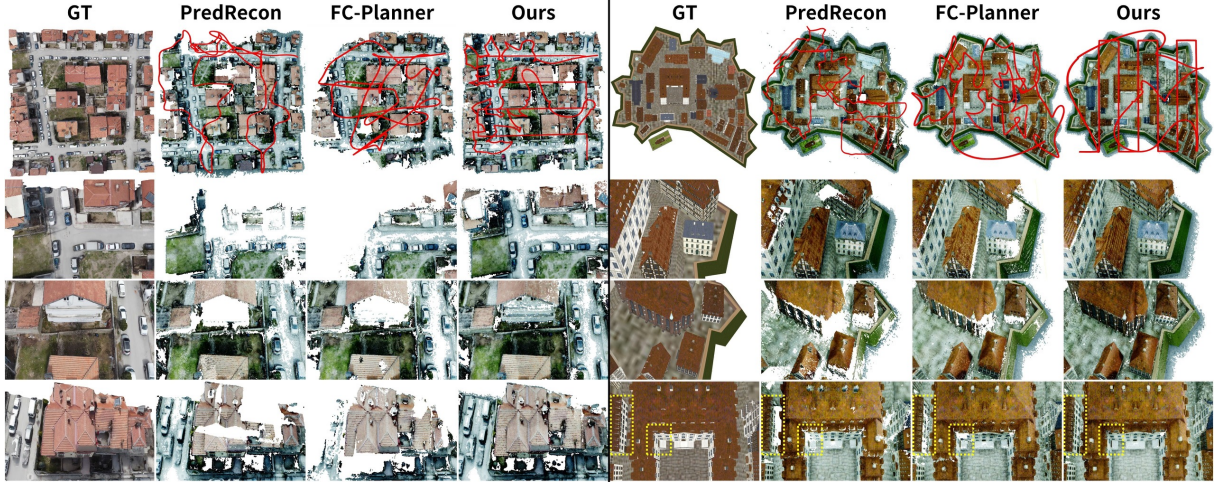


Fig. 4. Qualitative reconstruction results on **Turkey City** (left) and **EU City** (right) under the same travel budget. We compare PredRecon [11], FC-Planner [12], and our method against the ground truth; red curves indicate the planned flight trajectories. Our method yields denser and more complete surface reconstructions with fewer holes and cleaner roof/facade boundaries, as highlighted by the zoomed-in views.

remains saturated across all methods, performance variations are predominantly driven by Recall. As budgets tighten, the baseline Recall and F-scores drop noticeably. In contrast, our method degrades gracefully, consistently achieving the highest Recall and F-scores, with the performance gap peaking under the strictest constraints. For instance, in Atlas City ($B^* = 500$), our method yields an F-score of 85.93 and a Recall of 75.47, outperforming FC-Planner (80.77 and 67.95) and PredRecon (81.57 and 69.15). Furthermore, our method maintains high surface coverage across budgets, confirming that our online refinement effectively reallocates the travel budget to informative, under-observed regions without sacrificing geometric accuracy.

Beyond quantitative metrics, our approach offers practical advantages under fixed travel budgets. PredRecon performs continuous online modeling and replanning, which can impose substantial computational loads in onboard deployment. FC-Planner relies on a prior 3D model, yet its broad coverage trajectory may allocate budget to already-observed regions. In contrast, our prior-free method identifies coverage deficits online and reallocates the return budget to high-gain viewpoints, improving reconstruction completeness without requiring a pre-existing 3D model. Consequently, our planner focuses the constrained budget on informative revisits that improve Recall, outperforming the compared baselines.

Fig. 4 qualitatively supports these quantitative trends. Under identical budgets, PredRecon and FC-Planner leave noticeable holes and fragmented structures, particularly around occluded building facades and roofs. By explicitly targeting these under-observed areas during the return trajectory, our method generates denser, more complete reconstructions with cleaner outlines across all evaluated scenes.

C. Ablation Studies

To isolate the contribution of each module, we conduct controlled ablations by evaluating four variants. **Model-A** replaces the geometric visibility score S_{geo} with a simple

TABLE II

ABLATION RESULTS OF MODEL VARIANTS ON ATLAS CITY ($B^* = 700$).

Model	Description	Precision	Recall	F-score	Coverage
A	w/o Geometric score S_{geo}	99.78	78.40	87.79	98.85
B	w/o ALNS global update	99.72	77.63	87.30	97.79
C	w/o Sector-wise planning	99.84	79.01	88.21	97.29
D	Full system	99.83	78.97	88.18	98.71

TABLE III

COMPUTATION TIME OF EACH MODULE ON ATLAS CITY ($B^* = 700$).

Module	Online MVS (per frame)	Surf. Coverage	Sector Planner	ALNS Update (per iter)	Passing thru. a Sector
Time (sec)	0.56	52.04	7.71	0.11	95.24

visibility-count score, $\min(1, N_{\text{vis}}(x_j)/\tau_{\text{vis}})$, where $N_{\text{vis}}(x_j)$ denotes the number of observing views. **Model-B** disables the ALNS-based global update, relying solely on the sector planner. **Model-C** omits sector-wise planning, instead planning over the full scene in a batch manner. **Model-D** is our complete method. Table II summarizes the results.

Model-A yields lower Recall and F-scores, indicating that purely counting visible observations is insufficient for prioritizing informative views, whereas S_{geo} more effectively captures viewpoint quality. Model-B exhibits the largest degradation, indicating that sector-level routes alone are insufficient under tight budgets. Therefore, ALNS global refinement is crucial for reallocating the limited path length toward high-gain sectors and improving viewpoint selection beyond greedy aggregation. Model-D achieves nearly identical reconstruction quality to Model-C while distributing reconstruction and planning over the outbound flight, thereby avoiding the 310-second batch delay required by Model-C for 3D modeling and path planning.

Table III details the average runtimes of our online pipeline modules: (i) per-frame MVS processing, (ii) coverage evaluation, (iii) sector planning, (iv) ALNS iterations, and (v) UAV sector traversal. Crucially, the combined time for coverage

evaluation and sector planning is shorter than the UAV's sector traversal time, leaving an average slack of 35.49 seconds. The server leverages this background slack to execute approximately 320 ALNS global refinement iterations per sector while the UAV flies, seamlessly improving the route without introducing extra online latency.

V. CONCLUSIONS

We presented an online explore-then-exploit framework for budgeted UAV 3D reconstruction. Unlike previous methods relying on static priors, our approach dynamically identifies under-scanned regions and reallocates the return budget to high-gain viewpoints. Extensive evaluations show our method consistently achieves superior Recall and F-scores, especially under strict flight budgets. This demonstrates that dynamic, view-conditioned re-scanning is significantly more budget-efficient than prior-based optimized trajectories. Ultimately, our framework ensures complete, high-fidelity 3D modeling within a single flight, advancing practical autonomous photogrammetry.

REFERENCES

- [1] S. Song, D. Kim, and S. Jo, "Online coverage and inspection planning for 3d modeling," *Autonomous Robots*, vol. 44, no. 8, pp. 1431–1450, 2020.
- [2] R. Yu, P. Li, J. Shan, Y. Zhang, and Y. Dong, "Multi-feature driven rapid inspection of earthquake-induced damage on building facades using uav-derived point cloud," *Measurement*, vol. 232, 2024.
- [3] W. Xiong, B. Zeng, Z. Hu, J. Guo, K. Xie, and H. Huang, "Aerial path planning for urban geometry and texture co-capture," *ACM Transactions on Graphics (TOG)*, vol. 44, no. 6, pp. 1–17, 2025.
- [4] B. Hepp, M. Nießner, and O. Hilliges, "Plan3d: Viewpoint and trajectory optimization for aerial multi-view stereo reconstruction," *ACM Transactions on Graphics (TOG)*, vol. 38, no. 1, pp. 1–17, 2018.
- [5] M. Roberts, D. Dey, A. Truong, S. Sinha, S. Shah, A. Kapoor, P. Hanrahan, and N. Joshi, "Submodular trajectory optimization for aerial 3d scanning," in *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 5324–5333.
- [6] N. Smith, N. Moehle, M. Goesele, and W. Heidrich, "Aerial path planning for urban scene reconstruction: a continuous optimization method and benchmark," *ACM Trans. Graph.*, vol. 37, no. 6, 2018.
- [7] H. Zhang, Y. Yao, K. Xie, C.-W. Fu, H. Zhang, and H. Huang, "Continuous aerial path planning for 3d urban scene reconstruction," *ACM Trans. Graph.*, vol. 40, no. 6, pp. 225–1, 2021.
- [8] S. Song, D. Kim, and S. Choi, "View path planning via online multiview stereo for 3-d modeling of large-scale structures," *IEEE Transactions on Robotics*, vol. 38, no. 1, pp. 372–390, 2022.
- [9] S. Song, D. Kim, and S. Jo, "Active 3d modeling via online multi-view stereo," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 5284–5291.
- [10] Q. Kuang, J. Wu, J. Pan, and B. Zhou, "Real-time uav path planning for autonomous urban scene reconstruction," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [11] C. Feng, H. Li, F. Gao, B. Zhou, and S. Shen, "Predrecon: A prediction-boosted planning framework for fast and high-quality autonomous aerial reconstruction," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 1207–1213.
- [12] C. Feng, H. Li, M. Zhang, X. Chen, B. Zhou, and S. Shen, "Fc-planner: A skeleton-guided planning framework for fast aerial coverage of complex 3d scenes," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 8686–8692.
- [13] A. Santini, "An adaptive large neighbourhood search algorithm for the orienteering problem," *Expert Systems with Applications*, vol. 123, pp. 154–167, 2019.
- [14] S. Zhang, C. Liu, and N. Haala, "Guided by model quality: Uav path planning for complete and precise 3d reconstruction of complex buildings," *International Journal of Applied Earth Observation and Geoinformation*, vol. 127, p. 103667, 2024.
- [15] B. Englot and F. Hover, "Sampling-based coverage path planning for inspection of complex structures," in *Proceedings of the International Conference on Automated Planning and Scheduling*, vol. 22, 2012.
- [16] W. Jing, D. Deng, Z. Xiao, Y. Liu, and K. Shimada, "Coverage path planning using path primitive sampling and primitive coverage graph for visual inspection," in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2019, pp. 1472–1479.
- [17] I. D. C. Caiza, A. Milas, M. A. M. Grova, F. J. Perez-Grau, and T. Petrovic, "Autonomous exploration of unknown 3d environments using a frontier-based collector strategy," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [18] A. Batinovic, A. Ivanovic, T. Petrovic, and S. Bogdan, "A shadowcasting-based next-best-view planner for autonomous 3d exploration," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 2969–2976, 2022.
- [19] L. Schmid, M. Pantic, R. Khanna, L. Ott, R. Siegwart, and J. Nieto, "An efficient sampling-based method for online informative path planning in unknown environments," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 1500–1507, 2020.
- [20] R. Jin, Y. Gao, Y. Wang, Y. Wu, H. Lu, C. Xu, and F. Gao, "Gs-planner: A gaussian-splatting-based planning framework for active high-fidelity reconstruction," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024, pp. 11 202–11 209.
- [21] Y. Zhang, X. Chen, C. Feng, B. Zhou, and S. Shen, "Falcon: Fast autonomous aerial exploration using coverage path guidance," *IEEE Transactions on Robotics*, vol. 41, pp. 1365–1385, 2024.
- [22] Z. Sevkli and F. E. Sevilgen, "Variable neighborhood search for the orienteering problem," in *international symposium on computer and information sciences*. Springer, 2006, pp. 134–143.
- [23] D. Feillet, P. Dejax, and M. Gendreau, "Traveling salesman problems with profits," *Transportation science*, vol. 39, no. 2, 2005.
- [24] F. Hammami, "An efficient hybrid adaptive large neighborhood search method for the capacitated team orienteering problem," *Expert Systems with Applications*, vol. 249, p. 123561, 2024.
- [25] I. M. R. Fernández, C. E. Denniston, D. A. Caron, and G. S. Sukhatme, "Informative path planning to estimate quantiles for environmental analysis," *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 10 280–10 287, 2022.
- [26] Z. Wang, X. Zhou, C. Xu, and F. Gao, "Geometrically constrained trajectory optimization for multicopters," *IEEE Transactions on Robotics*, vol. 38, no. 5, pp. 3259–3278, 2022.
- [27] B. Lee, J. Park, K. T. Giang, S. Jo, and S. Song, "Mvs-gs: High-quality 3d gaussian splatting mapping via online multi-view stereo," *IEEE Access*, 2025.
- [28] B. Lee, J. Park, K. T. Giang, and S. Song, "Online 3d gaussian splatting modeling with novel view selection," in *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*, 2025.
- [29] Z. Teed and J. Deng, "Droid-slam: Deep visual slam for monocular, stereo, and rgb-d cameras," *Advances in neural information processing systems*, vol. 34, pp. 16 558–16 569, 2021.
- [30] C. Cao, X. Ren, and Y. Fu, "Mvsformer++: Revealing the devil in transformer's details for multi-view stereo," 2024.
- [31] S. Song, K. G. Truong, D. Kim, and S. Jo, "Prior depth-based multi-view stereo network for online 3d model reconstruction," *Pattern Recognition*, vol. 136, p. 109198, 2023.
- [32] R. B. Rusu and S. Cousins, "3d is here: Point cloud library (pcl)," in *2011 IEEE international conference on robotics and automation*. IEEE, 2011, pp. 1–4.
- [33] M. Kazhdan and H. Hoppe, "Screened poisson surface reconstruction," *ACM Transactions on Graphics (ToG)*, vol. 32, no. 3, pp. 1–13, 2013.
- [34] M. S. Ebeida, A. A. Davidson, A. Patney, P. M. Knupp, S. A. Mitchell, and J. D. Owens, "Efficient maximal poisson-disk sampling," *ACM Transactions on Graphics (TOG)*, vol. 30, no. 4, pp. 1–12, 2011.
- [35] E. Schubert, J. Sander, M. Ester, H. P. Kriegel, and X. Xu, "Dbscan revisited: why and how you should (still) use dbscan," *ACM transactions on database systems (tods)*, vol. 42, no. 3, pp. 1–21, 2017.
- [36] K. Helsgaun, "An extension of the lin-kernighan-helsgaun tsp solver for constrained traveling salesman and vehicle routing problems," *Roskilde: Roskilde University*, vol. 12, pp. 966–980, 2017.
- [37] J. L. Schönberger, E. Zheng, J.-M. Frahm, and M. Pollefeys, "Pixel-wise view selection for unstructured multi-view stereo," in *European conference on computer vision*. Springer, 2016, pp. 501–518.
- [38] A. Knapitsch, J. Park, Q.-Y. Zhou, and V. Koltun, "Tanks and temples: Benchmarking large-scale scene reconstruction," *ACM Transactions on Graphics (ToG)*, vol. 36, no. 4, pp. 1–13, 2017.